

ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

AIML Capstone Project Documentation

Supervised Learning & Unsupervised Learning Applications

Team Deep Mind

Email Spam Detection • Customer Persona Segmentation

Data preparation, machine learning, APIs, frontends, testing, collaboration, and integration

1 Introduction

The AIML Capstone Project brings together two machine learning applications that demonstrate different learning paradigms and the complete journey from data to an application workflow. The first application uses supervised learning to detect whether an email or message is Spam / Not Spam. The second uses unsupervised learning to group customers into meaningful personas based on Annual Income and Spending Score.

Together, the applications demonstrate more than isolated model training. Each application follows an end-to-end path:



The project was developed collaboratively using separate Git branches for different modules. Machine learning, backend, and frontend work was developed in the relevant branches, reviewed as a team, tested across module boundaries, and integrated into the main branch.

2 Project 1 – Email Spam Detection

2.1 Project Overview

Email Spam Detection is a text-classification application that identifies whether a supplied email or message should be treated as Spam or Ham / Not Spam. The application is useful because it provides an automated first-level classification of incoming text and demonstrates how natural-language data can be converted into features that a machine learning model can process.

This is a Supervised Machine Learning problem. The model is trained with labelled email data, where each record is associated with a known class. During prediction, the trained workflow processes new text in the same general way as the training data and returns a classification with confidence information.

2.2 What the Team Implemented

The Email Spam Detection workflow covers dataset preparation, text feature extraction, model training, evaluation, artifact saving, and application integration.



1. Dataset: The project uses spam_emails.csv as the source of labelled email/message data.
2. Data cleaning: Unnecessary characters are removed and the source text is standardized so that the input is more consistent for downstream processing.
3. Text preprocessing: Email text is prepared in a form suitable for feature extraction and classification.
4. TF-IDF feature extraction: Text is converted into numerical features using Term Frequency–Inverse Document Frequency vectorization.
5. Model training: The labelled data is split into training and testing sets, and a supervised model is trained to distinguish Spam from Ham.
6. Model evaluation: The workflow uses Accuracy, Precision, Recall, F1-Score, and a Confusion Matrix as evaluation methods. The supplied project information does not include numerical values for these measures, so none are reported here.
7. Saving artifacts: The trained model and TF-IDF vectorizer are saved so the application can reuse the learned workflow for new input.
8. Application integration: The saved artifacts are made available to the backend and frontend flow so a user can submit a message and view the result.

2.3 What the Application Can Do

- Accept an email or message as input.
- Process and standardize the text.
- Convert the text into numerical features using TF-IDF.
- Predict whether the email is Spam or Not Spam.
- Provide prediction and confidence information.
- Support integration with a frontend application.

2.4 Implementation

2.4.1 Dataset and Preprocessing

The source file is spam_emails.csv. The preprocessing stage cleans unnecessary characters, standardizes email text, and prepares the records for the supervised learning workflow. Consistent preprocessing is important because the same feature preparation logic must be applied to both training data and new messages submitted through the application.

2.4.2 Feature Extraction

TF-IDF Vectorization converts text into numerical features. It represents the importance of terms in relation to the email collection, allowing the classification model to work with a numerical representation of each message rather than raw text.

2.4.3 Model Training

The labelled dataset is divided into training and testing sets. The supervised learning model is trained to classify records into Spam and Ham classes. The saved training artifacts are later used by the application when a user submits a new message.

2.4.4 Model Evaluation

The team evaluated the model using Accuracy, Precision, Recall, F1-Score, and a Confusion Matrix. These measures provide complementary views of classification behaviour: overall correctness, class-specific correctness, coverage of positive cases, balance between precision and recall, and the distribution of correct and incorrect predictions.

2.4.5 Saved Model Files

File	Purpose
spam_model.pkl	Saved supervised classification model.
tfidf_vectorizer.pkl	Saved TF-IDF vectorizer used to transform text into model features.

3 Email Spam Detection – Team Contributions

Deepakkumar

Role: ML Developer – Email Spam Detection | Branch: spam-ml

Actual contribution:

- Prepared and cleaned spam_emails.csv.
- Preprocessed email text by removing unnecessary characters and standardizing the data.
- Converted text into numerical features using TF-IDF.
- Split the dataset into training and testing sets.
- Trained the machine learning model to classify Spam or Ham.
- Evaluated the model using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.
- Tested sample email inputs.

- Saved the trained model and TF-IDF vectorizer.

Files / modules: spam_emails.csv, train_model.py, spam_model.pkl, tfidf_vectorizer.pkl.

Workflow: Dataset → Data Cleaning → Text Preprocessing → TF-IDF → Model Training → Model Evaluation → Save Model → Application Integration.

Mahalakshmi Babu

Role: Backend Developer – Email Spam Detection | Branch: spam-backend

- Developed the FastAPI backend for the Email Spam Detection application.
- Implemented the /predict API endpoint for Spam / Not Spam prediction.
- Integrated the trained spam_model.pkl using Joblib.
- Added CORS and input validation using Pydantic.
- Developed the model training pipeline using TF-IDF and Multinomial Naive Bayes.
- Evaluated the model using Accuracy and Classification Report.
- Saved the trained model using Joblib.
- Tested the backend API with sample email inputs.

Files / modules: app.py, train_model.py, spam_emails.csv, and spam_model.pkl on spam-backend.

V. Tejasri Kotte

Role: Frontend Developer – Email Spam Detection | Branch: spam-frontend

- Created script.js.
- Connected the Check Email button with JavaScript.
- Read and validated email input.
- Sent email data to the FastAPI /predict endpoint using fetch().
- Processed the backend response.
- Displayed the Spam / Not Spam prediction and confidence percentage.
- Added loading status and error handling when the backend was unavailable.
- Used JSON to send email data and update prediction/confidence.
- Tested the application locally using Live Server.
- Verified input validation, button functionality, loading/error handling, and result display.
- Committed and pushed frontend changes to spam-frontend.

Files / modules: script.js and the related Email Spam Detection frontend on spam-frontend.

4 Project 2 – Customer Segmentation

4.1 Project Overview

Customer Segmentation is an Unsupervised Machine Learning application. It groups customers according to similarities in their characteristics without requiring predefined class labels. The project primarily uses Annual Income and Spending Score to identify meaningful customer personas.

The goal is to turn two customer attributes into interpretable groups that can support understanding of customer behaviour. The resulting clusters are mapped to the team's personas: Udhari, Pisanari, and Selavali.

4.2 Dataset Details

The customer dataset contains 600 records for both Annual Income and Spending Score. The statistics below are reproduced exactly from the project brief.

Statistic	Annual Income	Spending Score
Lowest value	15K	3.4
Highest value	120.8K	100
Count	600	600
Mean	68.380833	63.887167

Standard deviation	28.833117	28.681534
25%	34.075	30.575
Median	80.5	77.1
75%	90.7	86.125
Maximum	120.8	100

4.3 Customer Personas

Persona	Cluster interpretation
Udhari	Low income; high spending.
Pisnari	High income; low spending.
Selavali	High income; high spending score.

These personas are mapped from the K-Means clustering results. The cluster identifier produced by the model is interpreted through the project's persona mapping so that the application can present a meaningful customer type rather than only a numeric cluster label.

4.4 Implementation



1. **Dataset preparation:** Customer data is prepared using the **Annual Income** and **Spending Score** fields. The dataset is checked and organized before applying the clustering algorithm.
2. **Data preparation and validation:** The Annual Income and Spending Score fields are checked for valid numerical values and prepared for the machine learning workflow.
3. **StandardScaler:** The two numerical features are standardized using **StandardScaler** so that differences in feature scale do not disproportionately affect the clustering process.
4. **K-Means clustering:** The **K-Means clustering algorithm** is applied to group customers with similar income and spending characteristics.
5. **Three clusters:** The project uses **3 clusters** as the basis for identifying different customer groups.
6. **Persona mapping:** The three clustering outcomes are interpreted according to their income and spending characteristics and mapped to the project-defined personas:
 - **Udhari** – Low income and high spending.
 - **Pisnari** – High income and low spending.
 - **Selavali** – High income and high spending score.
7. **Model and scaler saving:** The trained **K-Means model** and **StandardScaler** are saved using **Joblib** so that the application can reuse them during live prediction without retraining.
8. **FastAPI backend:** A **FastAPI** backend is developed to provide health-check and prediction functionality. It loads the saved model and scaler and processes customer inputs received from the frontend.
9. **Streamlit frontend:** A **Streamlit** frontend is developed to accept customer inputs, communicate with the FastAPI prediction API, and display the predicted cluster and corresponding customer persona.
10. **Visualization:** A **2D Matplotlib visualization** is provided to represent the customer data and clustering results, helping users understand the distribution of the customer groups.
11. **Testing and integration:** The complete flow from customer input to API prediction and frontend result display is tested to verify that the ML model, backend, and frontend work together correctly.

5 Customer Segmentation – Team Contributions

Santosh R

Role: ML Developer – Customer Segmentation | Branch: customer-ml

- Worked on customer dataset generation and preparation.
- Prepared customer data for clustering.
- Checked Annual Income and Spending Score data.
- Identified dataset statistics, including the lowest and highest income and spending values.
- Applied StandardScaler.
- Implemented K-Means clustering using 3 clusters.
- Mapped clusters to customer personas.
- Defined Udhari as low income/high spend, Pisanari as high income/low spend, and Selavali as high income/high spending score.
- Saved the trained model and scaler using joblib.

Files / modules: dataset_unsupervised.py, train_kmeans.py, and customers.csv.

Dataset values recorded: Annual Income from 15K to 120.8K; Spending Score from 3.4 to 100; count 600 for each field.

Dyneshwaran V

Role: Backend Developer & ML Integrator – Customer Segmentation | Branch: customer-backend

- Developed the FastAPI backend.
- Worked on unsupervised/customer-segmentation/main_unsupervised.py.
- Implemented FastAPI/Uvicorn setup.
- Created the /health endpoint.
- Implemented the /predict endpoint for live inference.
- Used pathlib.Path anchored to __file__ to dynamically locate and load kmeans_model.pkl.
- Used joblib for model loading.
- Added input validation and exception handling.
- Handled missing or non-numeric input using appropriate HTTP 422 responses.
- Handled model loading failures using HTTP 503.
- Added CORSMiddleware to allow communication between Streamlit and FastAPI.
- Diagnosed Windows socket errors such as WinError 10013 and WinError 10048.
- Used fallback ports such as 8000/8001 when required.
- Managed the customer-backend Git branch and staged and pushed backend changes.

Files / modules: unsupervised/customer-segmentation/main_unsupervised.py and the Customer Segmentation backend module on customer-backend.

Arulmeenai A

Role: Data Preparation – Customer Segmentation | Branch: customer-ml

- Assisted with customer dataset preparation and cleaning.
- Helped organize and prepare the customer dataset for the ML workflow.
- Assisted with checking the Annual Income and Spending Score fields.
- Supported basic data validation before K-Means clustering.
- Helped ensure the dataset was ready for the machine learning stage.

Nihetha M

Role: Team Leader & Frontend Developer – Customer Segmentation | Branch: customer-frontend

- Allocated ML, backend, and frontend responsibilities among team members.
- Coordinated the development workflow between team members.
- Coordinated the ML → Backend → Frontend integration flow.

- Worked on Customer Segmentation frontend.
- Worked on the customer-frontend branch.
- Developed the Streamlit dashboard.
- Created the customer input interface.
- Connected Streamlit with the FastAPI backend using requests.
- Sent customer data to the /predict API.
- Received and processed the backend response.
- Displayed the predicted cluster and corresponding customer persona.
- Added 2D Matplotlib visualization.
- Tested the frontend and verified the complete Customer Segmentation flow.

GitHub and Integration Responsibilities

- Reviewed the work completed across the different team branches.
- Checked integration between ML models, backend APIs, and frontend applications.
- Verified that the different modules worked together correctly.
- Pushed the reviewed and integrated work to the main branch.
- Coordinated final testing and integration.

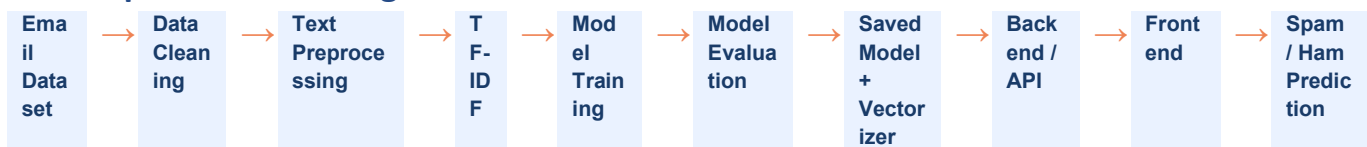
6 Technology Stack

The following technologies and tools were used to develop the project.

Area	Technologies / methods	Use in the project
Machine Learning	Python; Scikit-learn; TF-IDF; K-Means; StandardScaler; Joblib	Model training, text feature extraction, clustering, scaling, and saving/loading model artifacts.
Backend	FastAPI; Uvicorn; REST API; CORS	Prediction endpoints, health endpoint, local API serving, and communication between backend and frontend.
Frontend	HTML; CSS; JavaScript; Streamlit; Matplotlib	Email Spam Detection interface, browser-side API interaction, Customer Segmentation dashboard, and 2D visualization.
Development / collaboration	Git; GitHub; Git branches; VS Code	Branch-based development, collaboration, review, integration, and local development.

7 Overall Project Workflow

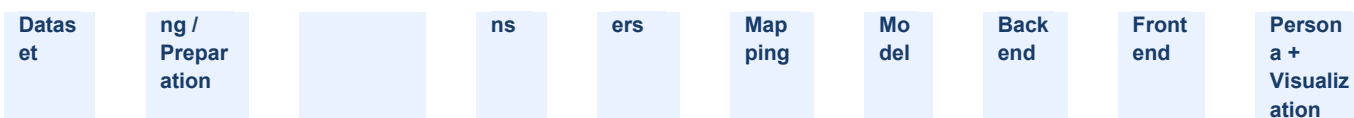
7.1 Supervised Learning Workflow



The supervised workflow begins with labelled email data and ends with a user-visible Spam / Not Spam prediction. The saved model and vectorizer bridge model development and application inference.

7.2 Unsupervised Learning Workflow





The unsupervised workflow begins with customer attributes rather than labelled classes. K-Means produces three clusters, which are then mapped to the project's customer personas and shown through the API and Streamlit application.

7.3 Team Workflow



8 Team Contributions Summary Table

Name	Role	Branch	Main contribution
Deepakkumar	ML Developer – Email Spam Detection	spam-ml	Prepared and cleaned the email dataset; implemented preprocessing, TF-IDF, model training/evaluation, sample testing, and saved the model artifacts.
Mahalakshmi Babu	Backend Developer – Email Spam Detection	spam-backend	Developed the FastAPI backend, implemented the /predict API, integrated the trained model, and worked on the TF-IDF and Multinomial Naive Bayes training pipeline.
V. Tejasri Kotte	Frontend Developer – Email Spam Detection	spam-frontend	Implemented JavaScript interaction, input validation, FastAPI /predict communication, response display, loading/error handling, testing, and branch push.
Santosh R	ML Developer – Customer Segmentation	customer-ml	Prepared customer data, checked statistics, applied StandardScaler, implemented 3-cluster K-Means, mapped personas, and saved model/scaler artifacts.
Dyneshwaran V	Backend Developer & ML Integrator – Customer Segmentation	customer-backend	Developed FastAPI/Uvicorn backend, health and prediction endpoints, model loading, validation, exceptions, CORS, port troubleshooting, and branch management.
Arulmeenai A	Customer Segmentation- data preparation support	customer-ml	Assisted with customer dataset preparation, cleaning, field checks, and validation before clustering
Nihetha M	Frontend Developer – Customer Segmentation	customer-frontend	Customer Segmentation Streamlit frontend + FastAPI integration + visualization + branch review + final integration review and push to main.